

Hands-on 4: Difference between JPA, Hibernate and Spring Data JPA

Aim

To understand the concepts of JPA, Hibernate, and Spring Data JPA, and learn how they are related in developing Java applications that interact with databases.

Introduction

In Java application development, storing and retrieving data from a database is a common requirement. Earlier, developers had to write a large amount of JDBC code to perform database operations, which made applications complex and difficult to maintain. To simplify this process, technologies like JPA, Hibernate, and Spring Data JPA were introduced.

1. Java Persistence API (JPA)

Java Persistence API, commonly known as JPA, is a specification provided by Java for managing data between Java objects and relational databases. It defines a standard set of rules and interfaces that developers can use for object-relational mapping (ORM). Since JPA is only a specification, it does not contain any implementation by itself.

The main objective of JPA is to provide a common standard so that developers can switch between different persistence providers without changing much of their application code. It uses annotations to map Java classes and their fields to database tables and columns.

Uses annotations like:

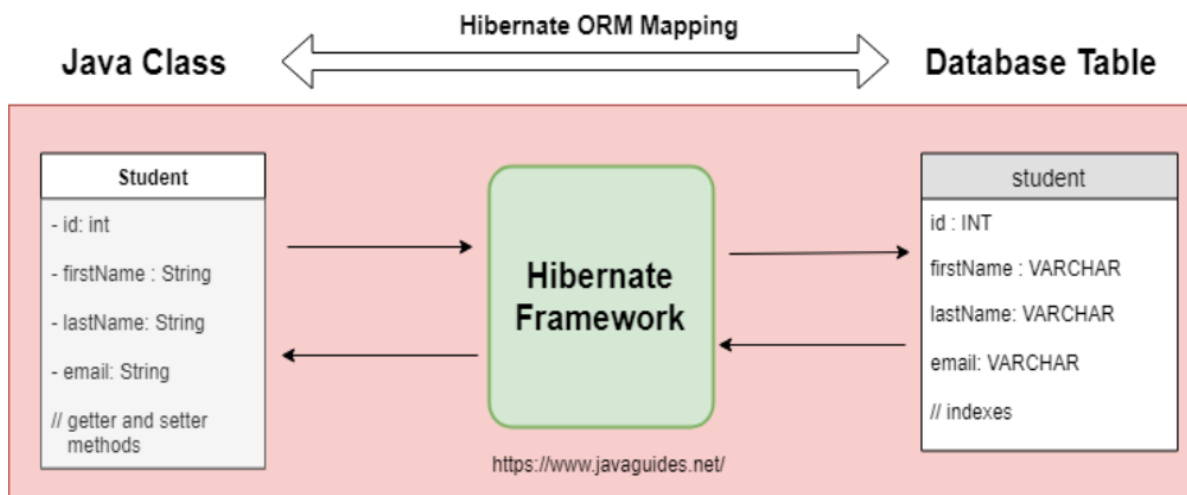
- `@Entity`
- `@Table`
- `@Id`
- `@Column`

2.What Is Hibernate Framework?

Hibernate is an object-relational mapping solution for Java environments. Object-relational mapping or ORM is the programming technique to map application domain model objects to the relational database tables.

Hibernate provides a reference implementation of the Java Persistence API that makes it a great choice as an ORM tool with the benefits of loose coupling.

Example: Below diagram shows an *Object Relational Mapping* between the **Student** Java class and the **students** table in the database.



3.What Is Spring Data JPA?

Spring Data is a part of the SpringFramework. The goal of Spring Data repository abstraction is to significantly reduce the amount of boilerplate code required to implement data access layers for various persistence stores.

Spring Data JPA is not a JPA provider. It is a library/framework that adds an extra layer of abstraction on the top of our JPA provider (like Hibernate).

Spring Data JPA automatically provides common database operations such as saving, updating, deleting, and retrieving records through repository interfaces. It also manages transactions automatically, allowing developers to focus more on business logic rather than database management.

Difference Between Hibernate and Spring Data JPA

Hibernate provides complete ORM functionality and gives developers greater control over database operations. However, it requires more coding for handling sessions, transactions, and queries.

Spring Data JPA is built on top of Hibernate and significantly reduces the amount of code required. Developers only need to define repository interfaces, while Spring automatically handles most database operations. This leads to cleaner, more maintainable, and less error-prone applications.

Relationship Between JPA, Hibernate and Spring Data JPA



Comparison Table

Hibernate Example

```
public Integer addEmployee(Employee employee){

    Session session = factory.openSession();

    Transaction tx = null;

    Integer employeeID = null;

    try {
```

```

        tx = session.beginTransaction();

        employeeID = (Integer) session.save(employee);

        tx.commit();
    } catch (HibernateException e) {
        if (tx != null)
            tx.rollback();

        e.printStackTrace();
    } finally {
        session.close();
    }

    return employeeID;
}

```

Spring Data JPA Example

EmployeeRepository.java

```

public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
}

```

EmployeeService.java

```

@Autowired

private EmployeeRepository employeeRepository;

@Transactional

public void addEmployee(Employee employee) {

```

```
employeeRepository.save(employee);  
}
```

Explanation

- Extends `JpaRepository`.
- No implementation code is required.
- `save()` method is already provided.
- Spring automatically manages transactions.
- No session handling is needed.

This makes the code cleaner and easier to maintain.

Conclusion

JPA is a specification that defines how Java objects should be persisted into relational databases. Hibernate is one of the most popular implementations of JPA and provides the actual ORM functionality. Spring Data JPA is a higher-level abstraction built on top of JPA that significantly reduces boilerplate code by providing repository interfaces and automatic CRUD operations. Therefore, modern Spring Boot applications commonly use **Spring Data JPA with Hibernate** because it simplifies development, improves maintainability, and minimizes manual coding.